

Benchmarking reproducible de modelos de detección de intrusiones en infraestructura local y Microsoft Azure: rendimiento, optimización y costo

Aiko Masaki Perales, Mildred Bautista Abanto, Jean Trujillo Dominguez y Sebastian Minchola Robles
Universidad del Pacífico, Lima, Perú
{AV.MasakiP, MB.BautistaA, JL.TrujilloD, WS.MincholaR}@alum.up.edu.pe

Resumen—La elección entre infraestructura local y nube para entrenar sistemas de detección de intrusiones no puede reducirse a asumir que la elasticidad —capacidad de ajustar recursos bajo demanda— implica menor tiempo. Este trabajo presenta un *benchmark*, entendido como una evaluación comparativa bajo un protocolo común, de Random Forest, XGBoost y un perceptrón multicapa sobre 700,001 registros de UNSW-NB15. Se evaluaron cuatro escenarios que cruzan entorno de ejecución (equipo local y VM Standard_D4s_v3 de Microsoft Azure) con dos pipelines (base y optimizado), usando cuatro núcleos lógicos, cinco semillas y 60 entrenamientos. El paquete de optimización combinó paralelización, reducción de memoria y ajustes específicos por modelo; por ello sus resultados representan un efecto conjunto y no el efecto causal de una técnica aislada. Random Forest y XGBoost redujeron el tiempo de entrenamiento entre 49.5 % y 59.5 % en ambos entornos. Los intervalos de confianza exploratorios muestran que la ventaja temporal de Azure frente al equipo local no fue concluyente para Random Forest ni XGBoost; solo el MLP optimizado mostró una diferencia favorable a Azure. XGBoost local optimizado alcanzó el menor tiempo medio (4.43 s) con F1 de 0.9767, mientras que Random Forest base obtuvo el mayor F1 (0.9792) y la menor tasa de falsos negativos. Los lotes completos costaron 0.0489 y 0.0321 USD en local, y 0.0499 y 0.0331 USD en Azure, para los pipelines base y optimizado, respectivamente. No existe un ganador universal: la elección depende de la prioridad asignada a calidad, tiempo y costo.

Index Terms—intrusion detection, machine learning, cloud computing, benchmarking, UNSW-NB15, XGBoost, reproducibilidad

I. INTRODUCCIÓN

La detección de intrusiones basada en aprendizaje automático combina dos exigencias: discriminar tráfico malicioso con pocos falsos negativos y ejecutar el ciclo de entrenamiento e inferencia con recursos razonables. El NIST Cybersecurity Framework 2.0 sitúa la identificación, detección y respuesta como funciones centrales de la gestión del riesgo digital [1]. Sin embargo, una buena métrica predictiva no resuelve por sí sola la decisión de infraestructura: un modelo puede ser preciso y, al mismo tiempo, costoso, variable o difícil de reproducir.

La nube ofrece aprovisionamiento bajo demanda y elasticidad para ajustar recursos según la carga, mientras que el hardware local utiliza capacidad ya disponible. La ventaja relativa depende del algoritmo, del paralelismo efectivo y de la arquitectura subyacente. La literatura sobre sistemas de detección de intrusiones (IDS), es decir, el conjunto de

estudios científicos de esta área, se concentra principalmente en exactitud y F1 [2], [3]; los trabajos de edge/cloud, por su parte, suelen estudiar inferencia, latencia o despliegues IoT [4], [5]. Queda así una brecha práctica: comparar un mismo experimento de IDS en infraestructura local y nube, preservando software, datos, semillas y presupuesto de concurrencia.

Este trabajo aborda el siguiente problema: *¿qué cambia realmente al ejecutar un mismo benchmark de entrenamiento de IDS en un equipo local y una VM cloud cuando ambos se limitan a cuatro núcleos lógicos, y cuánto aporta un pipeline optimizado frente al pipeline base?* La pregunta se descompone en tres preguntas de investigación (PI):

- **PI1:** ¿Cómo varían tiempo, estabilidad y uso de recursos entre el entorno local y Azure bajo el mismo límite de concurrencia?
- **PI2:** ¿Qué reducción temporal y qué compromiso predictivo produce el paquete de optimización en cada entorno?
- **PI3:** ¿Qué implicaciones de costo y operación se desprenden a la escala de 700,001 registros?

El objetivo general es **comparar empíricamente el rendimiento computacional, la calidad predictiva y el costo marginal de tres modelos de IDS en un entorno local y una VM Azure, con y sin un paquete de optimización**. No se pretende demostrar escalabilidad asintótica ni aislar una técnica individual de optimización.

Las contribuciones son: (i) un diseño 2×2 con cinco semillas y 60 entrenamientos; (ii) una comparación multidimensional que incluye tiempo, F1, falsos negativos, CPU, RAM y costo local/cloud; (iii) contrastes exploratorios con intervalos de confianza del 95 %; y (iv) una guía de decisión basada en calidad, tiempo y costo.

II. ESTADO DEL ARTE

Los antecedentes relevantes se organizan en tres ejes: calidad predictiva de IDS, reducción del costo computacional y benchmarking de infraestructura. Esta organización evita presentar trabajos relacionados como una lista y permite localizar con precisión la brecha cubierta.

II-A. IDS y datos desbalanceados

Ali *et al.* [2] sistematizan estrategias de ML para IDS y destacan que la comparación debe ir más allá de accuracy, especialmente en datasets desbalanceados. Ajagbe *et al.* [3]

comparan clasificadores sobre datos no balanceados y respaldan el uso de métricas como precisión, recall y F1. Estos estudios justifican la selección de modelos y métricas, pero no comparan un mismo pipeline entre infraestructura local y nube.

II-B. Eficiencia computacional de IDS

LocKedge [4] combina reducción de dimensionalidad y una red neuronal compacta para detección en edge, midiendo complejidad y consumo de recursos. Aldaej *et al.* [6] reducen atributos en un IDS IoT basado en RNN/Bi-LSTM y muestran que es posible disminuir demanda computacional conservando capacidad predictiva. Akinie *et al.* [7] distribuyen preentrenamiento y ajuste fino entre servidor y dispositivos edge. Los tres trabajos alinean seguridad con eficiencia, aunque sus escenarios edge/federados difieren de una comparación local–Azure de entrenamiento clásico.

II-C. Benchmarking edge y cloud

Carpio *et al.* [8] comparan experimentalmente arquitecturas cloud-only, edge-only y edge-cloud bajo carga variable, mostrando que no existe una arquitectura universalmente superior. EdgeInsight [5] caracteriza inferencia de ML en edge y cloud y demuestra que la configuración, el preprocesamiento y la arquitectura del modelo condicionan el resultado. ECBA-MLI [9] propone una arquitectura reproducible para evaluar latencia y energía en múltiples plataformas. Estos trabajos aportan el enfoque metodológico de benchmark, pero se orientan principalmente a inferencia y no a entrenamiento de IDS.

La brecha no consiste en afirmar que ningún trabajo haya comparado edge y cloud, sino en la falta de una comparación reproducible que combine IDS, entrenamiento, optimización, calidad predictiva y costo marginal bajo un mismo protocolo. Esa intersección define el alcance de este estudio.

III. METODOLOGÍA

III-A. Dataset y variable objetivo

Se utilizó UNSW-NB15[10], con 700,001 registros y 49 variables originales. La etiqueta binaria `Label` identifica tráfico normal o ataque. La distribución contiene 677,786 registros normales (96.83 %) y 22,215 ataques (3.17 %), por lo que accuracy aislada sería insuficiente. `attack_cat` se excluyó en ambos pipelines para evitar fuga directa de la clase.

Cada repetición empleó una partición estratificada 80/20 con semilla $42 + i$, $i \in \{0, \dots, 4\}$. El conjunto de prueba tuvo 140,001 observaciones. Las variables categóricas se imputaron y codificaron con *Label Encoding*; para el MLP, *StandardScaler* se ajustó solo con el conjunto de entrenamiento y luego se aplicó al conjunto de prueba.

III-B. Diseño experimental y escenarios

El diseño cruza dos factores operativos: entorno (local/Azure) y pipeline (base/optimizado). Los cuatro escenarios aparecen en la Tabla II. El término “optimizado” representa un **paquete de cambios**; no se interpreta como un tratamiento atómico.

Tabla II
ESCENARIOS DEL BENCHMARK

Escenario	Entorno	Pipeline
S1	Local	Base, ejecución secuencial
S2	Azure	Base, ejecución secuencial
S3	Local	Paquete optimizado
S4	Azure	Paquete optimizado

El pipeline base conserva `srcip` y `dstip`, no reduce tipos numéricos y configura Random Forest/XGBoost con `n_jobs=1`. El paquete optimizado elimina esas dos variables de alta cardinalidad, convierte puertos a formato numérico, reduce enteros y flotantes a tipos de menor tamaño, usa cuatro hilos en los modelos paralelizables, activa `tree_method=hist` en XGBoost y balanceo de clases en Random Forest. En el MLP, `max_iter` cambia de 50 a 200; no se observaron alertas de no convergencia. En consecuencia, S1→S3 y S2→S4 cuantifican el efecto conjunto del paquete, no el aporte separado de cada componente.

La memoria del dataframe pasó de 256.35 MB a 92.13 MB, una reducción de 64.06 %. Esta cifra corresponde a la representación del dataset, no a la memoria total del proceso.

III-C. Modelos, software e infraestructura

Se evaluaron Random Forest [11], XGBoost [12] y `MLPClassifier`, implementados con `scikit-learn` [13] y XGBoost. Cada combinación escenario–modelo se ejecutó cinco veces: $4 \times 3 \times 5 = 60$ entrenamientos.

Tabla III
INFRAESTRUCTURA EXPERIMENTAL

Característica	Local	Azure
CPU reportada	Intel Family 6 Model 170	VM Standard_D4s_v3
Núcleos físicos/lógicos	16/22	2/4 vCPU
Límite experimental	4 lógicos	4 vCPU
RAM total	31.32 GB	15.61 GB
Sistema operativo	Windows 11	Ubuntu/Linux Azure
Python	3.13.9	3.13.14
Región	Lima	Chile Central

El límite de cuatro núcleos controla el grado máximo de concurrencia, pero no iguala microarquitectura, frecuencia, sistema operativo, memoria, almacenamiento ni virtualización. Por ello, las diferencias se interpretan como desempeño de dos entornos concretos bajo igual presupuesto de concurrencia, no como efecto puro de “local” o “cloud”.

III-D. Métricas y análisis

Se midieron tiempo de entrenamiento, tiempo de inferencia batch, accuracy, precisión, recall, F1, CPU y RAM del sistema mediante `psutil`. La inferencia corresponde a `predict` sobre el conjunto de prueba y excluye transferencia WAN, serialización y exposición como servicio.

Se reportan media y desviación estándar. Además, se calcularon diferencias emparejadas por semilla y sus intervalos de confianza (IC) exploratorios del 95 % con distribución t ($n = 5$, 4 grados de libertad):

Tabla I
SÍNTESIS DE ANTECEDENTES Y BRECHA RESPECTO DEL PRESENTE ESTUDIO

Trabajo	Dominio	Entorno	Métricas principales	Diferencia frente a este estudio
Ajagbe <i>et al.</i> [3]	IDS/ML	Experimental, sin comparación de infraestructura	Accuracy, precisión, recall, F1	Cercano en tarea predictiva; no mide local frente a cloud ni costo computacional.
LocKedge [4]	IDS IoT	Edge y aprendizaje federado	Exactitud, complejidad, CPU/RAM	Cercano en eficiencia de IDS; prioriza despliegue edge e inferencia.
Aldaej <i>et al.</i> [6]	IDS IoT con DL	Edge-cloud propuesto, entrenamiento HPC	F1, recall, FLOPS, parámetros	Optimiza atributos y modelo; no realiza comparación simétrica local-cloud.
Carpio <i>et al.</i> [8]	Sistemas distribuidos	Cloud, edge y edge-cloud	Latencia, carga, CPU, memoria, red	Benchmark metodológico sólido; no usa ML ni ciberseguridad.
EdgeInsight [5]	Inferencia DL	Edge y cloud	Latencia y rendimiento de inferencia	Compara plataformas de ML; no estudia entrenamiento ni IDS.
Este trabajo	IDS/ML	Local y Azure, cuatro núcleos	Tiempo, F1, FN, CPU, RAM y costo	Mismo dataset, software y semillas; entrenamiento e inferencia batch en cuatro escenarios.

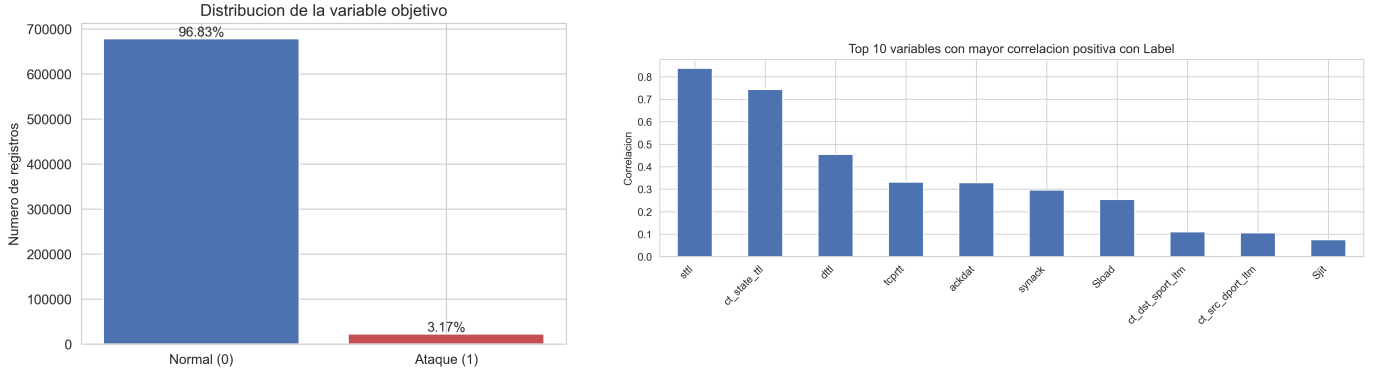


Figura 1. Distribución de la clase objetivo y variables numéricas con mayor correlación respecto de la etiqueta.

$$\bar{d} \pm t_{0.975,4} \frac{s_d}{\sqrt{5}}, \quad t_{0.975,4} = 2.776. \quad (1)$$

Aquí, $\bar{d}$ es la diferencia media entre dos configuraciones evaluadas con las mismas semillas, s_d es la desviación estándar de esas diferencias y $t_{0.975,4}$ es el valor crítico de Student. El IC delimita valores plausibles para la diferencia media: si incluye cero, los datos no permiten afirmar una ventaja temporal; si lo excluye, respaldan la dirección observada. Con cinco repeticiones, la interpretación es exploratoria y no equivale a demostrar igualdad cuando el intervalo contiene cero.

IV. RESULTADOS

IV-A. Rendimiento predictivo y temporal

La Tabla IV resume las 12 configuraciones. La infraestructura prácticamente no cambia las métricas predictivas cuando pipeline y semilla se mantienen; las diferencias de F1 se originan en el paquete de optimización y en la variación entre particiones.

IV-B. Contrastes con intervalos de confianza

La Tabla V expresa las diferencias como primera configuración menos segunda; valores positivos significan que la segunda fue más rápida. En Random Forest y XGBoost, el intervalo del efecto de optimización queda por encima de cero en ambos entornos. Para MLP, la dispersión impide

afirmar una reducción temporal consistente con solo cinco repeticiones.

Tabla V
DIFERENCIA MEDIA DE ENTRENAMIENTO E IC 95 % (s)

Contraste	Modelo	$\bar{d}$	IC 95 %
S1–S3 (optimización local)	RF	33.90	[16.69, 51.11]
	XGB	6.51	[1.61, 11.42]
	MLP	14.41	[−41.92, 70.74]
S2–S4 (optimización Azure)	RF	23.16	[20.77, 25.55]
	XGB	6.16	[3.75, 8.56]
	MLP	14.52	[−19.12, 48.15]
S1–S2 (infraestructura base)	RF	11.99	[−5.45, 29.43]
	XGB	−0.11	[−5.65, 5.43]
	MLP	18.74	[−1.17, 38.65]
S3–S4 (infraestructura opt.)	RF	1.25	[−5.57, 8.07]
	XGB	−0.47	[−1.64, 0.69]
	MLP	18.85	[0.61, 37.09]

Por ejemplo, el IC de S1–S3 para Random Forest, [16.69, 51.11] s, excluye cero y respalda una reducción por optimización local. En cambio, el IC de S3–S4 para XGBoost, [−1.64, 0.69] s, incluye cero y no permite concluir qué infraestructura fue más rápida. En consecuencia, PI1 no admite la regla “cloud siempre es más rápido”. Azure sí mostró menor desviación estándar en la mayoría de tiempos y una señal favorable para MLP optimizado.

Tabla IV
RESULTADOS POR ESCENARIO Y MODELO: MEDIA $\pm$ DESVIACIÓN ESTÁNDAR EN CINCO REPETICIONES

Escenario	Modelo	Entrenamiento (s)	Inferencia (s)	Precisión	Recall	F1	RAM sist. pico (GB)
S1 Local base	Random Forest	58.81 $\pm$ 14.15	0.498 $\pm$ 0.200	0.9721	0.9864	0.9792 $\pm$ 0.0012	19.13
	XGBoost	10.95 $\pm$ 3.35	0.283 $\pm$ 0.052	0.9735	0.9775	0.9755 $\pm$ 0.0063	19.13
	MLP	78.87 $\pm$ 31.64	0.384 $\pm$ 0.272	0.9476	0.9467	0.9470 $\pm$ 0.0042	19.20
S2 Azure base	Random Forest	46.82 $\pm$ 1.25	0.252 $\pm$ 0.011	0.9721	0.9864	0.9792 $\pm$ 0.0012	2.43
	XGBoost	11.06 $\pm$ 1.94	0.257 $\pm$ 0.002	0.9735	0.9775	0.9755 $\pm$ 0.0063	2.24
	MLP	60.12 $\pm$ 24.60	0.196 $\pm$ 0.121	0.9474	0.9473	0.9472 $\pm$ 0.0045	2.21
S3 Local optimizado	Random Forest	24.91 $\pm$ 5.78	0.210 $\pm$ 0.032	0.9724	0.9748	0.9736 $\pm$ 0.0014	18.80
	XGBoost	4.43 $\pm$ 0.92	0.131 $\pm$ 0.034	0.9751	0.9783	0.9767 $\pm$ 0.0008	18.67
	MLP	64.46 $\pm$ 16.67	0.145 $\pm$ 0.090	0.9328	0.9521	0.9423 $\pm$ 0.0018	18.65
S4 Azure optimizado	Random Forest	23.66 $\pm$ 0.92	0.162 $\pm$ 0.006	0.9724	0.9748	0.9736 $\pm$ 0.0014	2.56
	XGBoost	4.90 $\pm$ 0.05	0.146 $\pm$ 0.003	0.9751	0.9783	0.9767 $\pm$ 0.0008	2.43
	MLP	45.61 $\pm$ 9.63	0.122 $\pm$ 0.059	0.9328	0.9521	0.9423 $\pm$ 0.0018	2.42

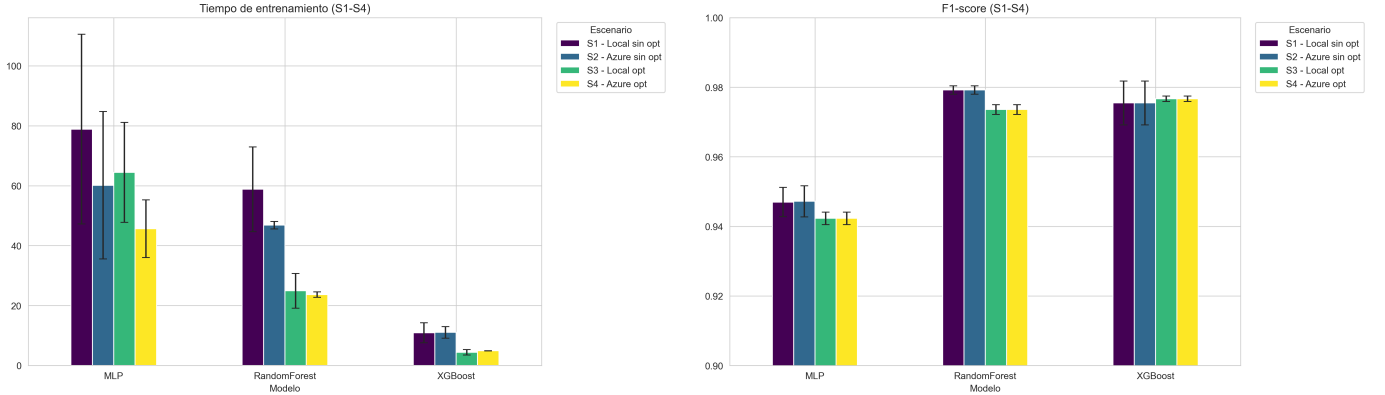


Figura 2. Tiempo de entrenamiento y F1 por escenario. La optimización domina la reducción temporal; el cambio de infraestructura no presenta una dirección universal.

IV-C. Calidad de detección y falsos negativos

Random Forest base obtuvo el mayor F1 (0.9792) y recall (0.9864). XGBoost optimizado combinó F1 0.9767 con el menor tiempo. El paquete de optimización redujo el F1 de Random Forest en 0.0056 (IC 95 % del cambio: [0.0035, 0.0077]) y aumentó su tasa de falsos negativos de 1.26 % a 2.81 % en la última repetición. Esta diferencia es pequeña en F1, pero relevante en seguridad: el número de ataques omitidos pasó de 56 a 125 sobre 4,443 ataques de prueba.

En XGBoost, el cambio medio de F1 fue favorable al pipeline optimizado en 0.0012, con un intervalo que incluye cero. Por tanto, no se sostiene una afirmación general de “optimización sin degradación”: el compromiso depende del modelo y de qué error sea más costoso.

IV-D. CPU, memoria y costo

Random Forest y XGBoost optimizados alcanzaron aproximadamente 100 % de CPU en Azure, coherente con cuatro trabajadores sobre cuatro vCPU. En local, `psutil` calcula porcentaje sobre los 22 hilos lógicos del sistema; por ello, un modelo limitado a cuatro trabajadores no debe alcanzar 100 % global. Las Figuras 4 deben interpretarse con esa diferencia de denominador.

La RAM registrada es memoria total usada por el sistema, no RSS (*Resident Set Size*), que representa la memoria física ocupada directamente por el proceso. La diferencia

absoluta local–Azure refleja también servicios residentes y estado previo del sistema; solo la reducción del dataframe (64.06 %) constituye una medición directamente atribuible al preprocesamiento.

El costo local incorpora electricidad y amortización del equipo:

$$C_{local} = \left(P_{equipo} C_{kWh} + \frac{C_{equipo}}{H_{vida}} \right) T, \quad (2)$$

donde P_{equipo} es la potencia en kW, C_{kWh} la tarifa eléctrica, C_{equipo} el precio de adquisición, H_{vida} la vida útil en horas y T el tiempo de ejecución en horas. Estos componentes producen la tasa equivalente documentada de 0.2175 USD/h. Para Azure se empleó 0.275 USD/h para Standard_D4s_v3 en Chile Central [14]. Los costos se obtienen multiplicando cada tasa por el tiempo total, incluidos 60 s de overhead por escenario.

Tabla VI
COSTO DEL LOTE EXPERIMENTAL POR ESCENARIO

Escenario	Tiempo (h)	USD/h	Costo (USD)
S1 Local base	0.22470	0.2175	0.04887
S3 Local optimizado	0.14762	0.2175	0.03211
S2 Azure base	0.18154	0.2750	0.04992
S4 Azure optimizado	0.12028	0.2750	0.03308

Los costos resultaron cercanos entre infraestructuras; la

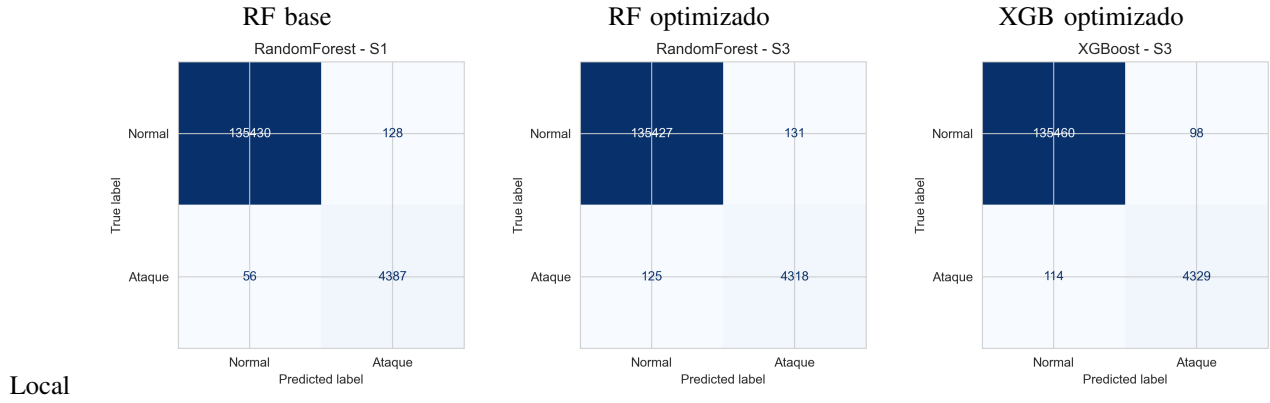


Figura 3. Matrices de confusión de la quinta repetición: mejor recall (RF base) frente a alternativas más rápidas.

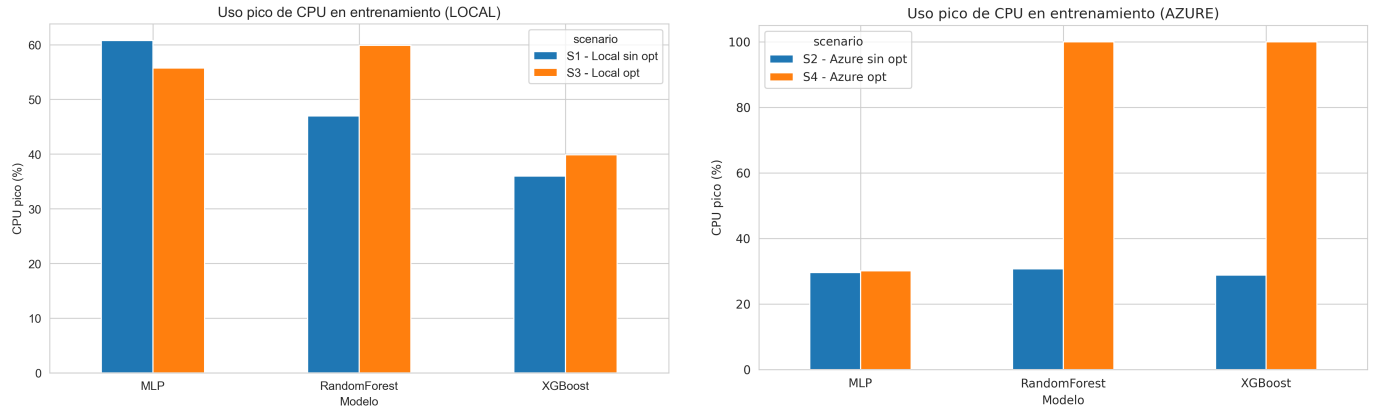


Figura 4. Uso de CPU reportado por el sistema. Los porcentajes no son directamente comparables porque el denominador es 22 hilos en local y 4 vCPU en Azure.

mayor diferencia provino de la optimización, que redujo el costo 34.3 % en local y 33.7 % en Azure.

V. DISCUSIÓN

V-A. Respuesta a las preguntas de investigación

P11 – infraestructura. Bajo cuatro núcleos lógicos, Azure no fue universalmente más rápido. Su ventaja más clara fue la menor variabilidad y el MLP optimizado; en XGBoost, el equipo local fue ligeramente más rápido. El resultado respalda lo observado en benchmarks edge/cloud: la plataforma ganadora depende de la carga y configuración [8], [5].

P12 – paquete de optimización. El efecto temporal fue más consistente que el cambio de infraestructura para Random Forest y XGBoost. El paquete combina paralelización, representación de datos e hiperparámetros, por lo que el resultado corresponde a su efecto conjunto. Además, Random Forest intercambió parte del ahorro temporal por más falsos negativos.

P13 – costo y operación. A esta escala, los cuatro lotes cuestan centavos y las diferencias local–Azure son pequeñas al incluir electricidad y amortización. XGBoost local optimizado ofrece el menor tiempo y costo por ejecución; Azure conserva ventajas operativas de aprovisionamiento, aislamiento y elasticidad, entendida como ajuste de recursos bajo demanda.

V-B. Guía de decisión

No existe un ganador claro en las tres variables: Random Forest base maximiza F1 y reduce falsos negativos, mientras que XGBoost local optimizado minimiza tiempo y costo con una disminución pequeña de F1. Por ello, en lugar de ocultar el compromiso dentro de un índice con pesos arbitrarios, la Tabla VII recomienda según la prioridad. El costo por ejecución se estimó con el tiempo medio total y la tarifa horaria, sin distribuir el overhead común del lote.

Tabla VII
GUÍA DE DECISIÓN SEGÚN CALIDAD, TIEMPO Y COSTO

Prioridad	Configuración recomendada y fundamento
Mejor balance tiempo–costo	S3–XGBoost: 4.43 s, F1 0.9767 y 0.00028 USD por ejecución media.
Mayor calidad de detección	Random Forest base: F1 0.9792, recall 0.9864 y 56 FN en la última repetición.
Mayor calidad con menor tiempo	S2–Random Forest: mismo F1 0.9792 que S1, 46.82 s frente a 58.81 s, con costo por ejecución casi igual.
Estabilidad temporal	Azure, por la menor dispersión observada en la mayoría de configuraciones.
Elasticidad y concurrencia	Azure cuando se requiera ajustar recursos o ejecutar cargas simultáneas.

V-C. Limitaciones y recomendaciones para trabajos futuros

Validez de medición. CPU y RAM describen el estado global de cada sistema, por lo que sirven para caracterizar

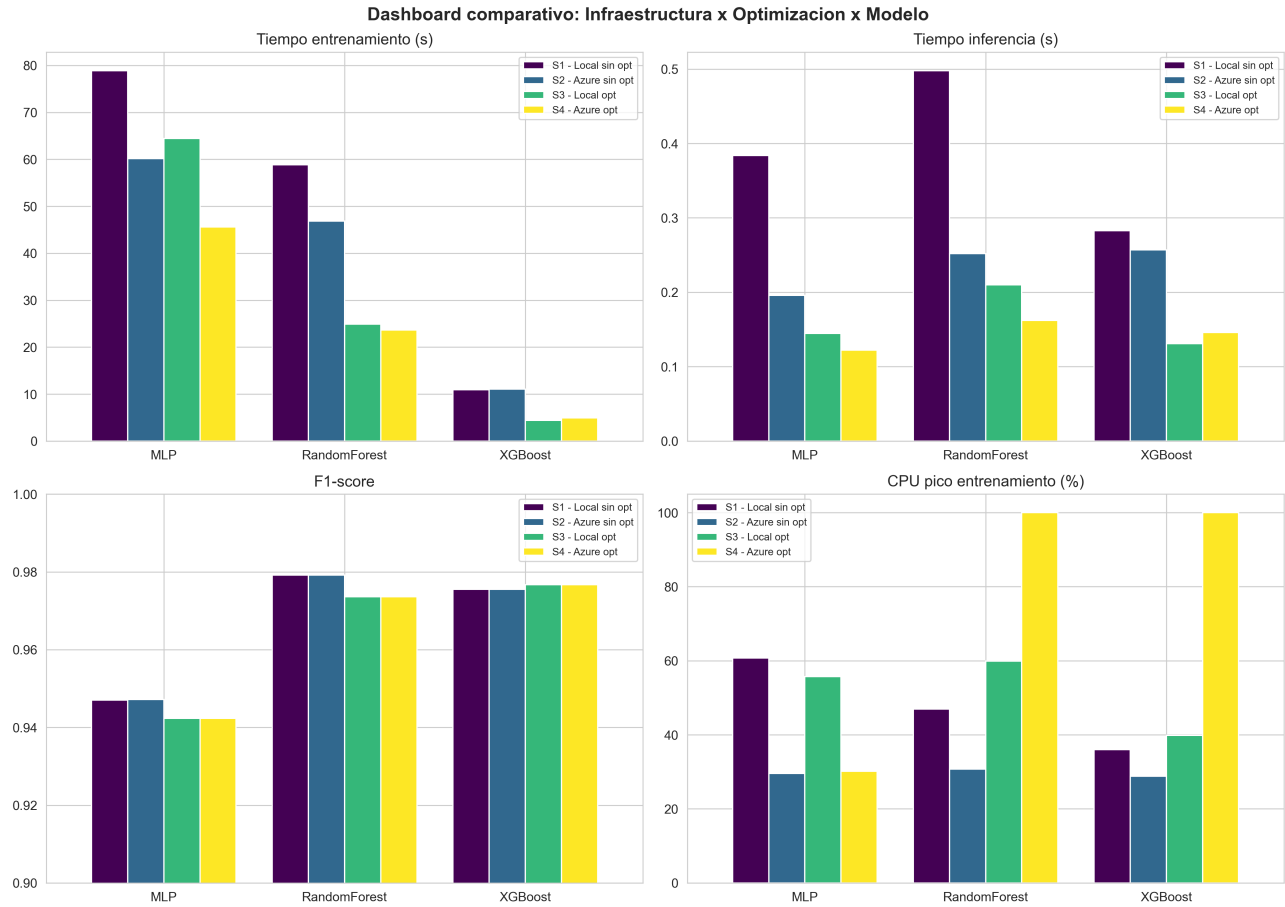


Figura 5. Vista consolidada del benchmark. La lectura conjunta evita seleccionar una configuración usando solo accuracy o solo tiempo.

cada host, pero no aíslan el consumo del proceso. Un trabajo futuro puede complementar estas métricas con RSS, energía y latencia de una API, además de promediar las matrices de confusión de todas las repeticiones.

Validez externa. Los resultados caracterizan una partición de UNSW-NB15, un equipo local y una familia de VM con cuatro núcleos. Para ampliar su aplicación se recomienda evaluar volúmenes crecientes, más familias y regiones de VM, GPU, entrenamiento distribuido y al menos 20 repeticiones por configuración. Este diseño permitiría estimar el punto de equilibrio entre costo, calidad y tiempo a otras escalas.

VI. CONCLUSIONES

El experimento muestra que la decisión entre local y cloud no admite una respuesta universal. Para Random Forest y XGBoost, el paquete de optimización redujo el tiempo de forma consistente en ambos entornos; el cambio de infraestructura produjo diferencias menores y con intervalos amplios. XGBoost local optimizado minimizó tiempo y costo, mientras que Random Forest base priorizó la detección de ataques al obtener el mayor F1, recall y menos falsos negativos.

La principal conclusión es metodológica: **antes de migrar una carga de IDS a la nube, conviene optimizar y medir el pipeline.** Al incorporar electricidad y amortización, los costos

local y Azure fueron similares; la optimización redujo ambos aproximadamente un tercio. La guía de decisión explicita el compromiso: S3–XGBoost ofrece el mejor balance tiempo–costo, mientras que Random Forest base es preferible si la prioridad es maximizar la detección.

Como trabajo futuro se propone un diseño de ablación que active una optimización por vez y un dashboard interactivo que permita explorar calidad, tiempo, costo y uso de recursos por escenario. El despliegue mediante Streamlit y una VM Azure facilitaría comunicar los resultados sin recargar el artículo con todas las figuras.

REFERENCIAS

- [1] National Institute of Standards and Technology, “The nist cybersecurity framework (csf) 2.0,” National Institute of Standards and Technology, Tech. Rep., 2024.
- [2] A. H. Ali, M. Charfeddine, B. Ammar, B. B. Hamed, F. Albalwy, A. Alqarafi, and A. Hussain, “Unveiling machine learning strategies and considerations in intrusion detection systems: A comprehensive survey,” *Frontiers in Computer Science*, vol. 6, 2024.
- [3] S. A. Ajagbe, J. B. Awotunde, and H. Florez, “Intrusion detection: A comparison study of machine learning models using unbalanced dataset,” *SN Computer Science*, vol. 5, no. 8, 2024.
- [4] T. T. Huong, T. P. Bac, D. M. Long, B. D. Thang, N. T. Binh, T. D. Luong, and T. K. Phuc, “LocKedge: Low-complexity cyberattack detection in IoT edge computing,” *IEEE Access*, vol. 9, pp. 29 696–29 710, 2021.

- [5] P. Ross and A. Luckow, "EdgeInsight: Characterizing and modeling the performance of machine learning inference on the edge and cloud," in *2019 IEEE International Conference on Big Data (Big Data)*, 2019, pp. 1897–1906.
- [6] A. Aldaej, T. A. Ahanger, and I. Ullah, "Deep learning-inspired IoT-IDS mechanism for edge computing environments," *Sensors*, vol. 23, no. 24, p. 9869, 2023.
- [7] R. Akinie, N. K. Gyimah, M. Bhavsar, and J. Kelly, "Fine-tuning federated learning-based intrusion detection systems for transportation IoT," in *SoutheastCon 2025*, 2025.
- [8] F. Carpio, M. Delgado, and A. Jukan, "Engineering and experimentally benchmarking a container-based edge computing system," in *ICC 2020 – 2020 IEEE International Conference on Communications*, 2020, pp. 1–6.
- [9] M. Schneider, R. Prokscha, S. Saadani, and A. Hoss, "ECBA-MLI: Edge computing benchmark architecture for machine learning inference," in *2022 IEEE International Conference on Edge Computing and Communications (EDGE)*, 2022, pp. 23–32.
- [10] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *2015 Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1–6.
- [11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. VanderPlas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [14] Microsoft Azure, "Linux virtual machines pricing," <https://azure.microsoft.com/pricing/details/virtual-machines/linux/>, 2026, tarifa observada en Azure Portal para Standard D4s v3, Chile Central, 19 de mayo de 2026.